PX4 + ROS 2 con px4_examples

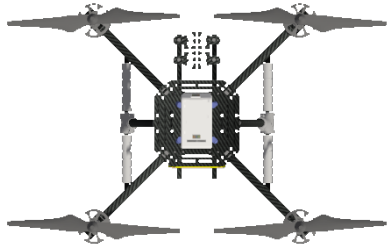
Paso a paso: de instalar a volar el x500 con tu propio nodo (Parte 2)

Imad Jared Cabrera Trejo · Vespo UAV · 11 de noviembre de 2026



Hoy

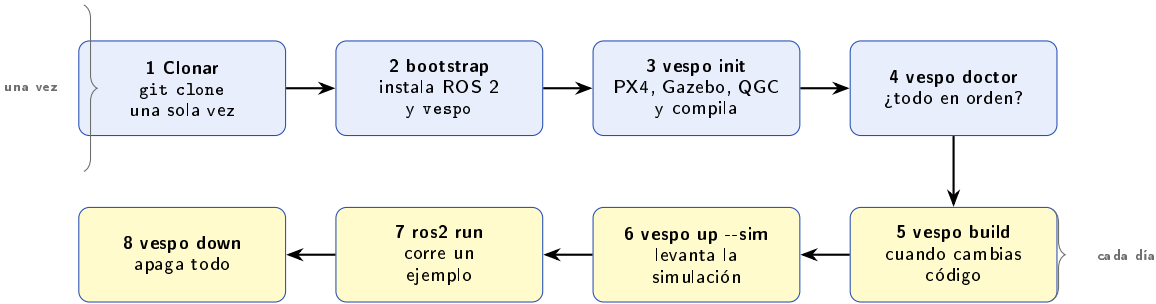
1. **Instalar** todo con `vespo`, desde cero.
2. **Repaso** del x500 y de PX4 en 4 láminas.
3. Cómo **habla** ROS 2 con PX4.
4. `shared_ptr`: lo único de C++ que hoy hace falta.
5. **Escuchar** al dron.
6. **Volar en offboard**: `square_mission` cuadro por cuadro.
7. **Volar con comandos**: `goto_mission` y `return_home`.
8. **Tu primer reto**: la estrella del show.
9. **Cuando algo no jala**.



Contenido

1. Instalar todo con vespo
2. Repaso: el x500 y PX4
3. Cómo habla ROS 2 con PX4
4. shared_ptr: lo único de C++ que hoy hace falta
5. Escuchar al dron
6. Volar en offboard
7. Volar con comandos
8. Tu primer reto: la estrella del show
9. Cuando algo no jala

El camino completo



Los pasos **1 a 4** se hacen una vez por computadora. Del **5 al 8** es lo de todos los días.

Antes de empezar



Lo que necesitas

- **Ubuntu 24.04** (no otra versión: es la única con ROS 2 Jazzy).
- **~30 GB libres**: PX4 y Gazebo solos ocupan ~15 GB.
- **Buen internet**: la primera vez baja varios GB.
- Tu **contraseña** (pide sudo para instalar).
- **1 a 2 horas** la primera vez.

Conecta el cargador

La simulación usa mucho CPU y tarjeta de video. Con batería la laptop se pone lenta y puede **apagarse sola** a media simulación.

Si algo se corta (internet, batería), **no pasa nada**: vuelves a correr el mismo comando y sigue donde iba.

Paso 1: clonar el repositorio




```
sudo apt install git                                  # si no lo tienes
cd ~                                                  # en tu carpeta personal
git clone https://github.com/Imcab/vespo_ws.git
cd vespo_ws
```

~/ (tu carpeta)

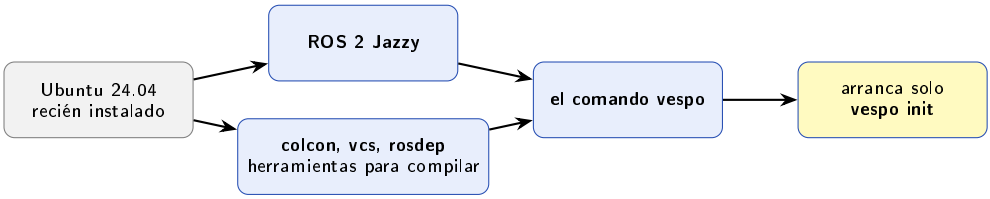
- `vespo_ws/` el workspace del equipo
  - `src/` los paquetes de ROS 2
  - `tools/` el comando vespo
  - `vespo.toml` versiones del equipo
- `PX4-Autopilot/` lo baja el paso 3

- Un **workspace** es la carpeta con todos los paquetes de ROS 2 del equipo.
- PX4 vive **al lado**, no adentro.
- `vespo.toml` fija la versión de PX4 (**v1.17.0**): todos usan la misma.

Paso 2: bootstrap.sh




```
./bootstrap.sh          # dentro de vespo_ws; te pide tu contraseña
```



- Es lo **único** que se corre a mano en una máquina nueva: lo demás lo hace `vespo`.
- Al terminar le pasa el control a `vespo init` (el paso 3), que te va a hacer preguntas.
- Si se corta: `./bootstrap.sh` otra vez.

Paso 3: vespo init, las preguntas



`vespo init` pregunta en inglés. Para una laptop de simulación:

Primero, cómo es tu máquina

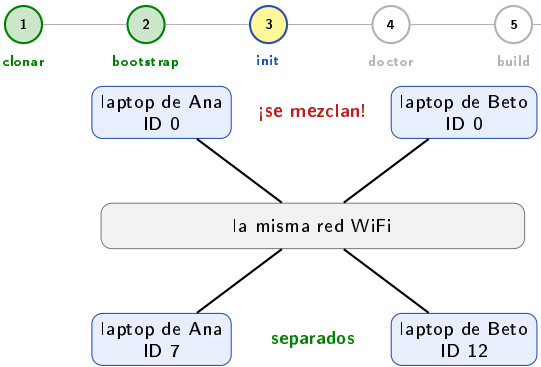
Te pregunta	Contesta
`Role of this machine`	**sim** (tu laptop simula)
`ROS_DOMAIN_ID`	**tu número** (siguiente lámina)
`Flight controller`	**None yet** (no hay Pixhawk)
`Path to PX4-Autopilot`	**Enter** (al lado de `vespo_ws`)
Simulation defaults	**Enter** a todo

Luego, qué instalar: Y a todo

Te pregunta	Hace
`Clone them now with vcs?`	baja `px4_msgs` y el agente
`Install them now in ...?`	PX4 y Gazebo, ~**15 GB**
`Install missing ROS dependencies...?`	pide contraseña
`Build the workspace now?`	compila
`Build PX4 SITL now?`	compila PX4
`Download QGroundControl now?`	la estación de tierra
`Add vespo... to ~/.bashrc?`	cada terminal lo tiene

Termina con `sim machine ready`. Si lo vuelves a correr, tus respuestas anteriores salen por defecto.

ROS_DOMAIN_ID: cada quien su canal



- En la misma red, los nodos de ROS 2 de **distintas laptops se ven entre sí**.
- Si dos personas simulan con el mismo ID, tu nodo puede leer (**y mandarle órdenes**) al dron de otro.
- Cada quien usa **su número** (0 a 232).
- Cambiarlo después: `vespo config edit domain_id`.

Paso 4: terminal nueva y vespo doctor


```
# abre una terminal NUEVA (la vieja
# todavia no conoce el comando vespo)
vespo doctor
```

Revisa todo lo que se instaló y te dice si falta algo. Úsalo **siempre que algo no funcione**: es lo primero.

Junto a cada ✗ viene un `fix:` con el comando que lo arregla.

vespo doctor

```
System ----------------------
✓ ROS 2 jazzy installed
✓ colcon found
Workspace -------------------
✓ Role: sim
✓ px4_msgs at the pinned version
PX4 -------------------------
✓ PX4-Autopilot at v1.17.0
✓ px4_msgs matches PX4's messages
✓ PX4 SITL built
✓ Gazebo Harmonic 8.15.0
Link to PX4 -----------------
✓ UDP port 8888 free
QGroundControl --------------
✓ Installed at .../QGroundControl...
✓ All good
```

Si doctor marca algo en rojo



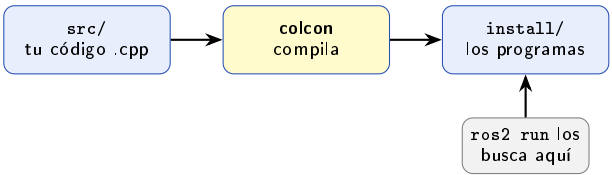
Dice	Corre	Qué pasó
`PX4-Autopilot is not at the pinned v1.17.0`	`vespo px4 sync`	PX4 en otra versión (p. ej. clonado de main)
`px4_msgs differs from PX4`	`vespo px4 sync`	los mensajes no coinciden: los topics llegarían mal
`px4_msgs not built`	`vespo build`	falta compilar
`px4_msgs not cloned`	`vespo init`	se saltó la clonada
`Workspace not in ~/.bashrc`	`vespo init`	contestaste n a la última pregunta
`ROS_DOMAIN_ID is ... in this shell`	abre otra terminal	la terminal es de antes del cambio
`UDP port 8888 busy`	`vespo down`	quedó un agente corriendo
`QGroundControl: Not installed`	`vespo qgc install`	no se bajó QGC

Ninguna de estas se arregla reinstalando todo: corre el `fix` que dice y vuelve a correr `vespo doctor`.

Paso 5: vespo build

1 clonar — 2 bootstrap — 3 init — 4 doctor — 5 build — 6 up — 7 ejemplo — 8 down

```
vespo build                  # todo el workspace
vespo build px4_examples     # solo un paquete (mucho mas rapido)
```



¿Cuándo?

- Cada vez que **cambias un .cpp**: C++ no corre el código nuevo hasta compilarlo.
- Después de `vespo update` no hace falta: ya compila solo lo que cambió.

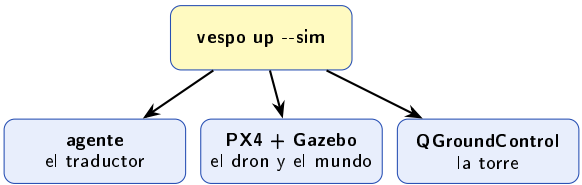
Ojo

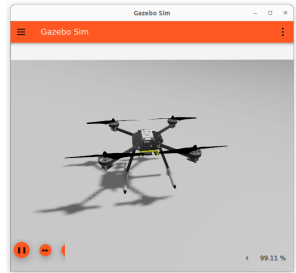
¿Cambiaste el código y el dron hace lo mismo de antes? Casi siempre es que **no compilaste**, o que el build falló y no viste el error.

Paso 6: vespo up --sim

1 clonar — 2 bootstrap — 3 init — 4 doctor — 5 build — 6 up — 7 ejemplo — 8 down

```
vespo up --sim
```





Gazebo con el x500. Documentación de PX4 (CC BY 4.0)

Espera a que QGC diga **Ready To Fly** (~30 s).

En la terminal **kitty** cada programa sale en su propio panel. En otra terminal salen todos juntos, cada línea con el nombre del programa al inicio. **Ctrl+C** los para a todos.

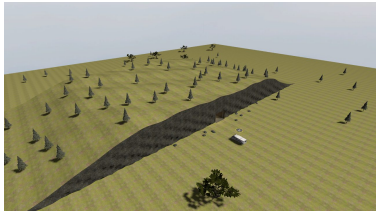
Variantes de vespo up

1 clonar — 2 bootstrap — 3 init — 4 doctor — 5 build — 6 up — 7 ejemplo — 8 down

```
vespo up --sim --world plaza_show                  # en otro mundo (los de x500_retos)
vespo up --sim --custom x500_lidar_2d              # con otro modelo de dron
vespo up --sim --world cueva --custom x500_lidar_2d
vespo up --sim --headless                          # sin ventana de Gazebo (mas ligero)
vespo simulate --show                              # que modelos y mundos hay
```



plaza_show



cueva



x500_lidar_2d

Paso 7: correr un ejemplo




```
# en OTRA terminal (vespo up sigue corriendo)
ros2 run px4_examples telemetry_listener
```

- `ros2 run` *paquete programa*.
- Tab completa los nombres: `ros2 run px4_examples <Tab><Tab>` lista los 18 ejemplos.
- Lo paras con **Ctrl+C**.

`ros2 run px4_examples telemetry_listener`

```
Estado: desarmado | modo: HOLD (4)
Local: x=0.01 y=-0.02 alt=0.03 m
| vel=(0.00, 0.00, 0.00) m/s
GPS: lat=47.3979711 lon=8.5461637
Actitud: roll=0.1° pitch=-0.2° yaw=90.0°
Batería: 16.20 V | 100 %
```

Si ves esto, **todo funciona**: ROS 2 ya habla con PX4.

Paso 8: apagar




```
vespo down          # desde cualquier terminal
```

- Para todo lo que levantó `vespo up`, y cualquier PX4, Gazebo o agente que haya quedado prendido.
- Úsalo también si la simulación se trabó: después `vespo up --sim` otra vez.

Procesos zombis

Si cierras la ventana de la terminal en vez de Ctrl+C, Gazebo o PX4 pueden quedarse corriendo escondidos. El siguiente `vespo up` choca con ellos ("port 8888 busy"). `vespo down` los limpia.

El día a día

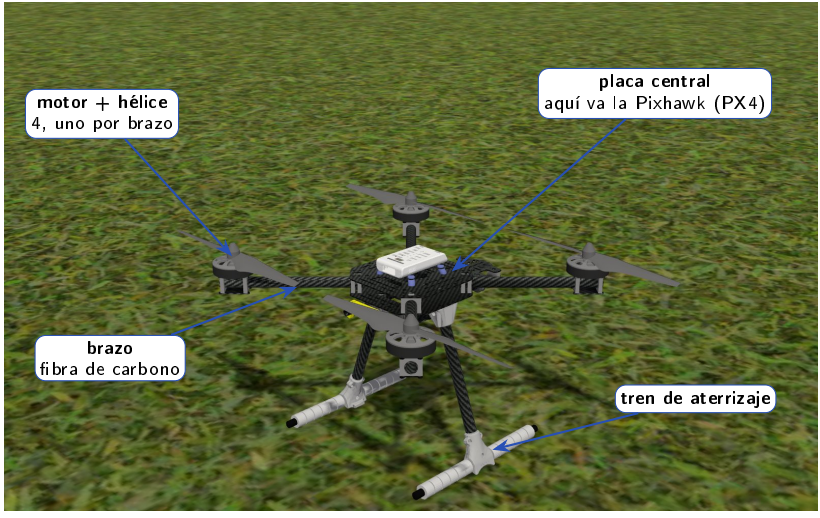
```
vespo update                    # lo nuevo del equipo
vespo up --sim                  # terminal 1
ros2 run px4_examples ...       # terminal 2
# ... editas un .cpp ...
vespo build px4_examples        # terminal 2
ros2 run px4_examples ...       # otra vez
vespo down                      # al terminar
```

- vespo te **avisa solo** cuando el equipo subió algo nuevo.
- No hace falta reiniciar la simulación para probar otra versión de tu nodo: Ctrl+C al nodo, compilas y lo corres de nuevo.
- `vespo config`: ver la configuración de tu máquina.

Contenido

1. Instalar todo con vespo
2. Repaso: el x500 y PX4
3. Cómo habla ROS 2 con PX4
4. shared_ptr: lo único de C++ que hoy hace falta
5. Escuchar al dron
6. Volar en offboard
7. Volar con comandos
8. Tu primer reto: la estrella del show
9. Cuando algo no jala

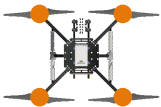
El x500 en una lámina



El x500 de la simulación de PX4. Render propio de Gazebo

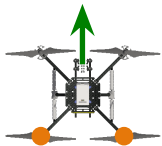
Cómo se mueve

Subir



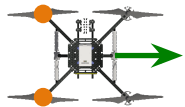
los 4 motores más rápido

Avanzar



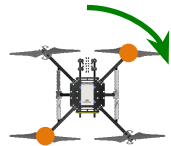
se inclina hacia adelante: los de atrás más rápido

Ir a la derecha



se inclina a la derecha: los de la izquierda más rápido

Girar (yaw)



acelera un par diagonal y frena el otro

- = motor que acelera. El frente del dron es **arriba** en los dibujos. Nunca le dices a PX4 qué motor acelerar: le dices **a dónde ir** y él lo calcula.

Los modos de vuelo: quién manda



Hold (nav 4)
se queda quieto en el aire

Offboard (nav 14)
obedece a **tu nodo**

Takeoff (nav 17)
despega a una altura

Land (nav 18)
aterriza donde está

Return (nav 5)
regresa a casa y aterriza

Position (nav 2)
un piloto lo vuela
con el control

Mission (nav 3)
sigue waypoints
hechos en QGC

Orbit (nav 21)
da vueltas a un punto

El modo actual sale en QGC y en `vehicle_status_v1.nav_state`. Si tu nodo "no hace nada", lo primero es ver **en qué modo está**.

Las reglas del piloto



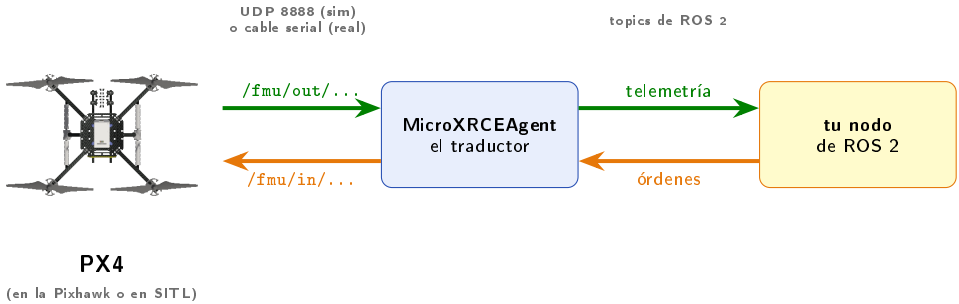
PX4 **siempre** está a cargo, aunque tu nodo esté mandando.

1. **No arma** si algo no está bien: sin GPS, con el estimador sin converger, sin QGroundControl conectado... QGC dice la razón.
2. **Armado** = motores girando. Si no despega en unos segundos, se **desarma solo**.
3. En **Offboard**, si tu nodo deja de mandar mensajes ~0.5 s, pasa a **Hold**.
4. Con **batería baja** regresa a casa (Return) solo.
5. Desde QGC **siempre** se le puede quitar el control a tu nodo: Hold, Land, Return.

Contenido

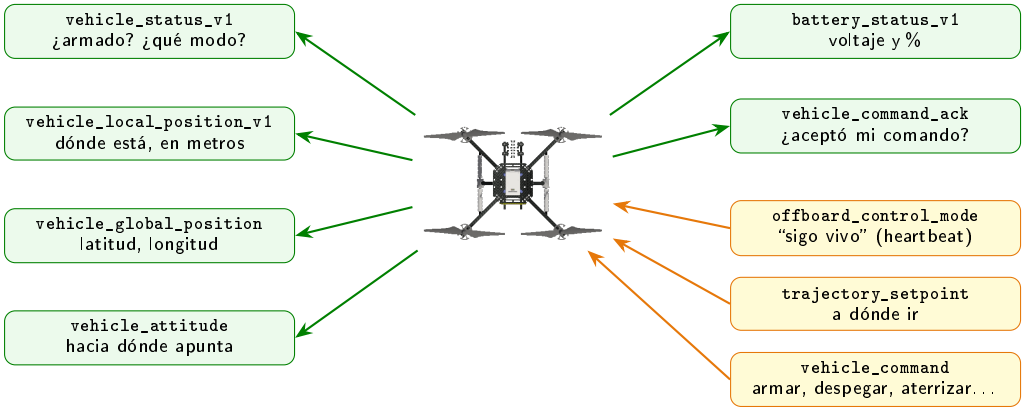
1. Instalar todo con vespo
2. Repaso: el x500 y PX4
3. Cómo habla ROS 2 con PX4
4. shared_ptr: lo único de C++ que hoy hace falta
5. Escuchar al dron
6. Volar en offboard
7. Volar con comandos
8. Tu primer reto: la estrella del show
9. Cuando algo no jala

Tres programas en fila



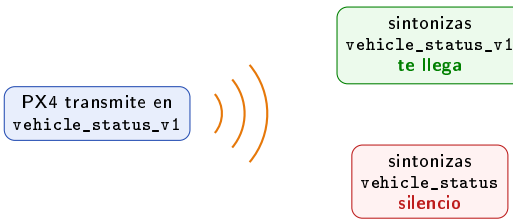
- PX4 no habla ROS 2 directamente: el **agente** traduce. `vespo up` lo arranca solo.
- **Sin agente no existe ningún topic** `/fmu/...` en ROS 2.
- `out`: PX4 te cuenta (verde). `in`: tú le pides (naranja).

El mapa de topics



Todos empiezan con `/fmu/out/` (verde) o `/fmu/in/` (naranja). Con estos nueve se hacen los 18 ejemplos.

La trampa del _v1



- Cuando PX4 cambia un mensaje, el nombre del topic cambia: _v1, _v2...
- Si te equivocas de nombre **no sale ningún error**: tu nodo se queda esperando para siempre.
- Copia los nombres de los ejemplos, o de `ros2 topic list | grep fmu`.

QoS: radio en vivo o carta certificada

best_effort = radio en vivo

Si un mensaje se pierde, ya pasó: el siguiente llega en milisegundos. **Así publica PX4**.

reliable = carta certificada

Se reenvía hasta que llega. Es lo que usa ROS 2 si no dices nada.

PX4 publica	tu nodo pide	¿llega?
best_effort	best_effort	✓
best_effort	reliable	×

Por eso todos los ejemplos empiezan con:

```
auto qos = rclcpp::QoS(1)
  .best_effort().transient_local();
```

Espiar desde la terminal

Con `vespo up --sim` corriendo, sin escribir código:

```
ros2 topic list | grep fmu                                  # que publica PX4
ros2 topic echo /fmu/out/vehicle_status_v1 --field nav_state  # el modo, en vivo
ros2 topic hz /fmu/out/vehicle_local_position_v1            # cada cuanto llega
ros2 interface show px4_msgs/msg/TrajectorySetpoint         # los campos de un mensaje
```

ros2 topic echo ... -field nav_state

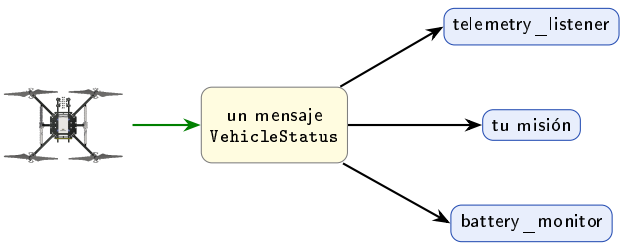
```
4
--
14
--
```

Cambia el modo desde QGC y míralo cambiar: **4** = Hold, **14** = Offboard, **18** = Land. Es la forma más rápida de saber si PX4 te está haciendo caso.

Contenido

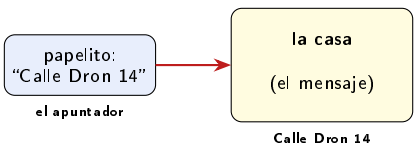
1. Instalar todo con vespo
2. Repaso: el x500 y PX4
3. Cómo habla ROS 2 con PX4
4. shared_ptr: lo único de C++ que hoy hace falta
5. Escuchar al dron
6. Volar en offboard
7. Volar con comandos
8. Tu primer reto: la estrella del show
9. Cuando algo no jala

El problema: un mensaje, muchos interesados



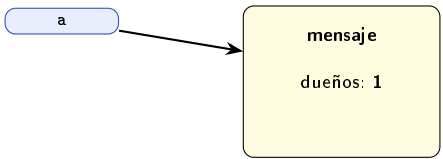
- Llega un mensaje y lo quieren **varias** partes del programa.
- ¿Copiarlo para cada una? Lento.
- ¿Compartir el original? Entonces, **¿quién lo borra** y cuándo? Si se borra antes de tiempo, el programa truena; si nadie lo borra, se acaba la memoria.

Primero: un apuntador es una dirección



- El mensaje vive en un lugar de la memoria (la **casa**).
- Un **apuntador** es un papelito con la dirección. Copiar el papelito es barato; copiar la casa no.
- `msg->nav_state`: ve a esa dirección y lee `nav_state`.
- `*msg`: la casa completa.

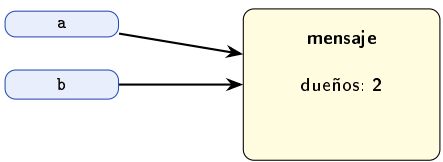
shared_ptr: la casa que se borra sola




```
auto a = std::make_shared<VehicleStatus>();  ◀
{
  auto b = a;      // copia el papelito
  {
    auto c = a;
  }  // c deja de existir
}    // b deja de existir
// a deja de existir: nadie lo usa
```

Se crea el mensaje. Una variable lo apunta: **1 dueño**.

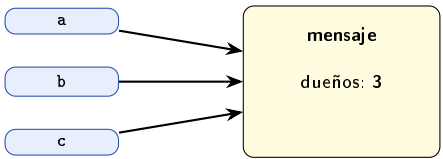
shared_ptr: la casa que se borra sola




```
auto a = std::make_shared<VehicleStatus>();
{
  auto b = a;      // copia el papelito  ◀
  {
    auto c = a;
  }  // c deja de existir
}    // b deja de existir
// a deja de existir: nadie lo usa
```

Copiar un `shared_ptr` copia la dirección, no el mensaje. Ahora son **2**.

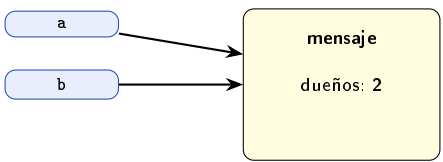
shared_ptr: la casa que se borra sola




```
auto a = std::make_shared<VehicleStatus>();
{
  auto b = a;      // copia el papelito
  {
    auto c = a;  ◀
  }  // c deja de existir
}    // b deja de existir
// a deja de existir: nadie lo usa
```

Otro más: **3** dueños. Los tres ven el **mismo** mensaje.

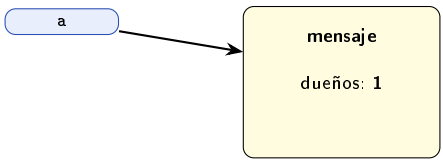
shared_ptr: la casa que se borra sola




```
auto a = std::make_shared<VehicleStatus>();
{
  auto b = a;      // copia el papelito
  {
    auto c = a;
  }  // c deja de existir
}    // b deja de existir
// a deja de existir: nadie lo usa
```

Cuando c sale de sus llaves, deja de existir y la cuenta baja.

shared_ptr: la casa que se borra sola




```
auto a = std::make_shared<VehicleStatus>();
{
  auto b = a;      // copia el papelito
  {
    auto c = a;
  }  // c deja de existir
}    // b deja de existir ◀
// a deja de existir: nadie lo usa
```

Lo mismo con b.

shared_ptr: la casa que se borra sola




```cpp
auto a = std::make_shared<VehicleStatus>();
{
  auto b = a;      // copia el papelito
  {
    auto c = a;
  }  // c deja de existir
}    // b deja de existir
// a deja de existir: nadie lo usa  ◀
```

El último dueño desaparece: la cuenta llega a 0 y el mensaje **se borra solo**. Nadie tuvo que acordarse.

Por eso en ROS 2 todo es SharedPtr

Los mensajes
`VehicleStatus::SharedPtr msg`
se entregan sin copiar

El nodo
`std::make_shared<Nodo>()`
vive mientras gira

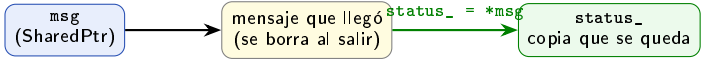
Publishers, subs, timers
`...::SharedPtr pub_`
existen mientras los guardes

`VehicleStatus::SharedPtr` es solo otro nombre para `std::shared_ptr<VehicleStatus>`. Cuando lo veas, piensa: “un papelito con la dirección de un `VehicleStatus`”.

En los ejemplos: guardar una copia con *msg

```
VehicleStatus status_;                          // la última copia, para usarla después

status_sub_ = create_subscription<VehicleStatus>(
  "/fmu/out/vehicle_status_v1", qos,
  [this](const VehicleStatus::SharedPtr msg) {    // msg = el papelito
    status_ = *msg;                               // *msg = el mensaje: se copia
  });
```



El callback **guarda**; el timer, cada 100 ms, **decide** con lo último que guardó. Así están hechos todos los ejemplos.

El error clásico: la suscripción que se muere

MAL: nunca recibe nada

```
Nodo() : Node("nodo") {
  auto sub = create_subscription<...>(...);
}  // sub era local: aqui se borra
   // y la suscripcion con el
```

BIEN: como en los ejemplos

```
rclcpp::Subscription<...>::SharedPtr sub_;
Nodo() : Node("nodo") {
  sub_ = create_subscription<...>(...);
}  // sub_ es del nodo: vive con el
```

Es el `shared_ptr` en acción: cuando el último dueño desaparece, el objeto se borra. Guarda publishers, subscriptions y timers en variables **del nodo** (las que terminan en _).

Resumen: punto, flecha y asterisco

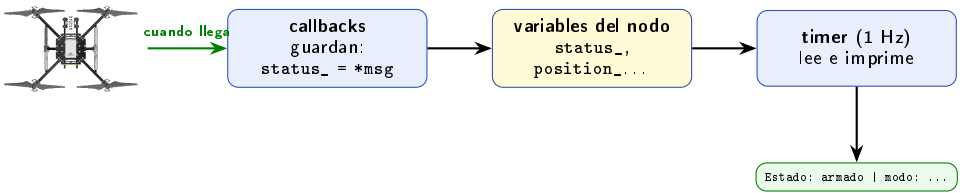
Escribes	Cuando tienes	Significa
`status_.nav_state`	un mensaje (`VehicleStatus`)	el campo, directo
`msg->nav_state`	un apuntador (`SharedPtr`)	ve a la dirección y lee el campo
`*msg`	un apuntador	el mensaje completo
`pub_->publish(m)`	un publisher (`SharedPtr`)	llamar una función a través del apuntador

Regla rápida: si la variable es un `SharedPtr`, usa `->`; si es el objeto, usa `.`. El compilador te avisa si te equivocas.

Contenido

1. Instalar todo con vespo
2. Repaso: el x500 y PX4
3. Cómo habla ROS 2 con PX4
4. shared_ptr: lo único de C++ que hoy hace falta
5. Escuchar al dron
6. Volar en offboard
7. Volar con comandos
8. Tu primer reto: la estrella del show
9. Cuando algo no jala

Todos los que escuchan tienen la misma forma



- Los mensajes llegan a 50–250 Hz: si imprimes en el callback, la terminal se vuelve ilegible.
- Por eso el callback solo **guarda** y un timer **imprime** una vez por segundo.
- Ninguno de estos nodos manda órdenes: son seguros de correr **siempre**, incluso con el dron real.

telemetry_listener: todo de un vistazo

ros2 run px4_examples telemetry_listener

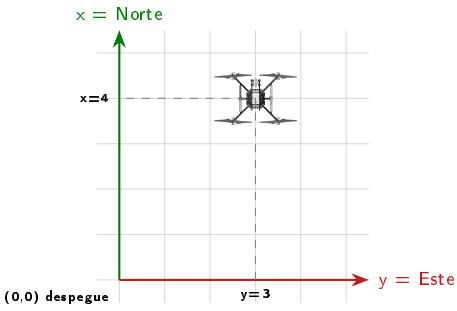
```
Estado: armado | modo: OFFBOARD (14)
Local: x=4.98 y=0.02 alt=5.01 m
| vel=(0.01, 1.98, 0.00) m/s
GPS: lat=47.3980159 lon=8.5461640
alt=5.1 m (AMSL)
Actitud: roll=2.1° pitch=0.4° yaw=0.2°
Batería: 15.40 V | 71 %
```

Ejemplo durante `square_mission`, yendo al este.

Estado	`vehicle_status_v1`
Local	`vehicle_local_position_v1`
GPS	`vehicle_global_position`
Actitud	`vehicle_attitude`
Batería	`battery_status_v1`

Pruébalo: con el dron en el aire, **muévelo desde QGC** y mira cómo cambian `x`, `y`, `alt` y el `yaw`. Así se aprende NED sin fórmulas.

Leer las coordenadas: NED



- `x`: metros al **norte** del despegue.
- `y`: metros al **este**.
- `z`: metros hacia **abajo**. Volando a 5 m, `z = -5`. Por eso los ejemplos imprimen `alt = -z`.
- `yaw`: hacia dónde apunta la nariz. 0° = norte, 90° = este.

Ojo

En Gazebo los ejes son otros (x al este, z arriba). Lo que importa para tu nodo es NED.

flight_mode_monitor: el narrador

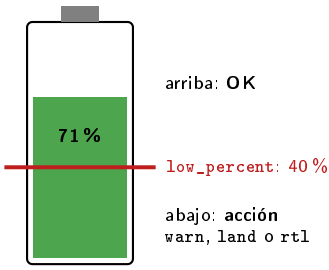
ros2 run px4_examples flight_mode_monitor

```
DESARMADO
Modo: Hold
Chequeos de pre-vuelo: OK, se puede armar
ARMADO
Modo: Hold -> Offboard
Modo: Offboard -> Hold
FAILSAFE ACTIVO (modo Hold)
Banderas de failsafe:
señal offboard perdida
```



Solo imprime cuando **algo cambia**: armado, modo, failsafe. Déjalo corriendo en una terminal **siempre** que pruebes algo: te dice qué hizo PX4 y por qué.

battery_monitor: tu propio failsafe




```
battery_monitor -p low_percent:=40.0 -p
action:=rtl
```

```
85.0% | 15.80 V (3.95 V/celda, 4S)
| -1.0 A | OK
BATERÍA BAJA: 39.8% (umbral 40%)
Regresando a casa (RTL)
```

- Escucha `battery_status_v1` y, bajo el umbral, **manda un comando** a PX4.
- En SITL la batería se descarga **rapidísimo** a propósito (~50 % en medio minuto armado) para poder probar esto.

Los cinco que escuchan

Ejemplo	Escucha	Para qué
`telemetry_listener`	estado, posición, GPS, actitud, batería	ver todo de un vistazo
`flight_mode_monitor`	estado, failsafes	saber qué hizo PX4 y por qué
`battery_monitor`	batería	actuar con batería baja
`imu_listener`	`sensor_combined`	giroscopio y acelerómetro crudos
`gps_status`	`vehicle_gps_position`	satélites, precisión, ¿listo para volar?

```
ros2 run px4_examples imu_listener      # quieto marca z = -9.8: la gravedad
ros2 run px4_examples gps_status
```

Contenido

1. Instalar todo con vespo
2. Repaso: el x500 y PX4
3. Cómo habla ROS 2 con PX4
4. shared_ptr: lo único de C++ que hoy hace falta
5. Escuchar al dron
6. Volar en offboard
7. Volar con comandos
8. Tu primer reto: la estrella del show
9. Cuando algo no jala

La receta de offboard: tres ingredientes

1
Heartbeat
`offboard_control_mode`
"sigo vivo y voy a controlar por posición"
10 veces por segundo

+

2
Setpoint
`trajectory_setpoint`
"quiero estar en (x, y, z) mirando a yaw"
10 veces por segundo

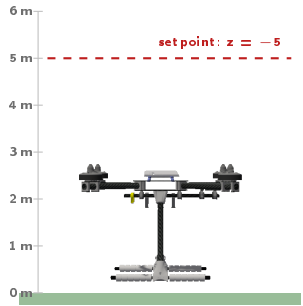
+

3
Modo + armar
`vehicle_command`
"entra a Offboard"
"arma los motores"
una vez, tras 1 s



- Sin **1** y **2** corriendo **antes**, PX4 rechaza entrar a Offboard: por eso el nodo "calienta" 1 s.
- Si **1** y **2** se detienen en pleno vuelo, PX4 pasa a Hold (failsafe).
- Los cinco ejemplos de offboard siguen esta receta; solo cambia **qué setpoint** mandan.

takeoff_mission paso a paso

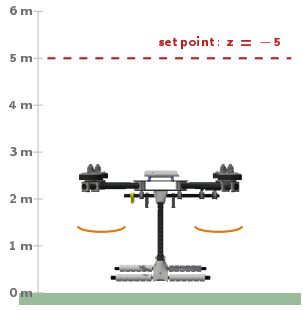



```
ros2 run px4_examples takeoff_mission
```

```
takeoff_mission iniciado, calentando setpoints...
arming=1 nav=4 alt=0.00 m
```

0 a 1 s, **calentar**: cada 100 ms publica el heartbeat y el setpoint {0, 0, -5}. El dron todavía está desarmado en tierra.

takeoff_mission paso a paso

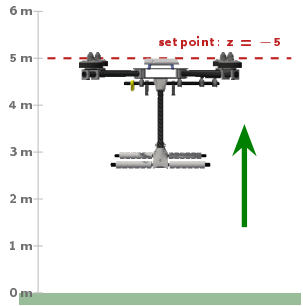


takeoff_mission

Comandos OFFBOARD + ARM enviados

A 1 s (counter_ == 10): pide modo Offboard y armar. Los motores arrancan.

takeoff_mission paso a paso

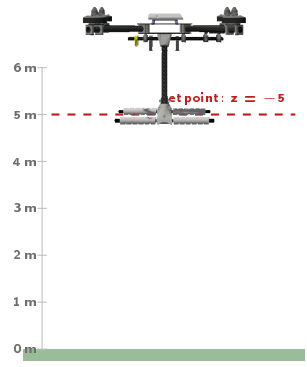


takeoff_mission

```
arming=2 nav=14 alt=2.47 m
```

Sube: PX4 ya está en Offboard (`nav=14`) y persigue el setpoint de 5 m. Tu nodo sigue mandando lo mismo; PX4 hace todo el trabajo.

takeoff_mission paso a paso

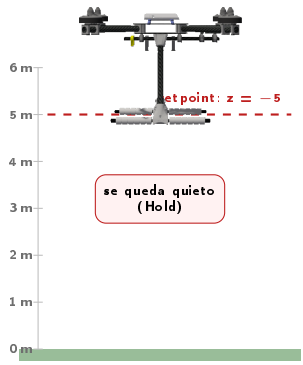


takeoff_mission

```
arming=2 nav=14 alt=5.00 m
```

Llegó: se queda a 5 m mientras el nodo siga publicando. Para siempre, si nadie lo para.

takeoff_mission paso a paso



flight_mode_monitor

```
Modo: Offboard -> Hold
FAILSAFE ACTIVO (modo Hold)
```

Ctrl+C al nodo: dejan de llegar setpoints, PX4 lo nota y pasa a **Hold**. No se cae. Para bajarlo: **Land** en QGC.

El corazón del nodo: loop() cada 100 ms

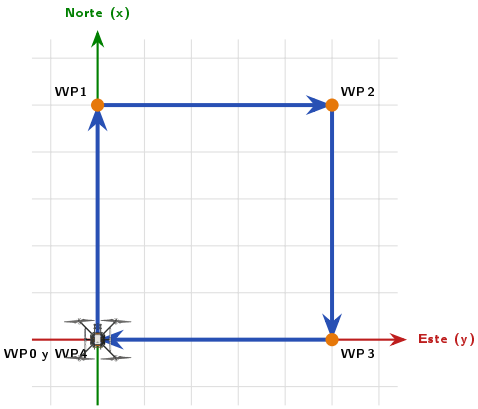
```
void loop() {
  OffboardControlMode offboard_msg{};                    // 1) heartbeat
  offboard_msg.position = true;
  offboard_msg.timestamp = now_us();
  offboard_pub_->publish(offboard_msg);

  TrajectorySetpoint setpoint_msg{};                     // 2) setpoint
  setpoint_msg.position = {0.0f, 0.0f, -5.0f};           //    norte, este, abajo
  setpoint_msg.yaw = 0.0f;                               //    mirando al norte
  setpoint_msg.timestamp = now_us();
  setpoint_pub_->publish(setpoint_msg);

  if (counter_ == 10) {                                  // 3) a 1 s: modo + armar
    send_command(VehicleCommand::VEHICLE_CMD_DO_SET_MODE, 1.0f, 6.0f);     // 6 = Offboard
    send_command(VehicleCommand::VEHICLE_CMD_COMPONENT_ARM_DISARM, 1.0f);  // 1 = armar
  }
  if (counter_ <= 10) counter_++;
}
```

Los tres ingredientes, en orden. Fíjate en el `->`: `offboard_pub_` es un `SharedPtr`.

square_mission: el plan

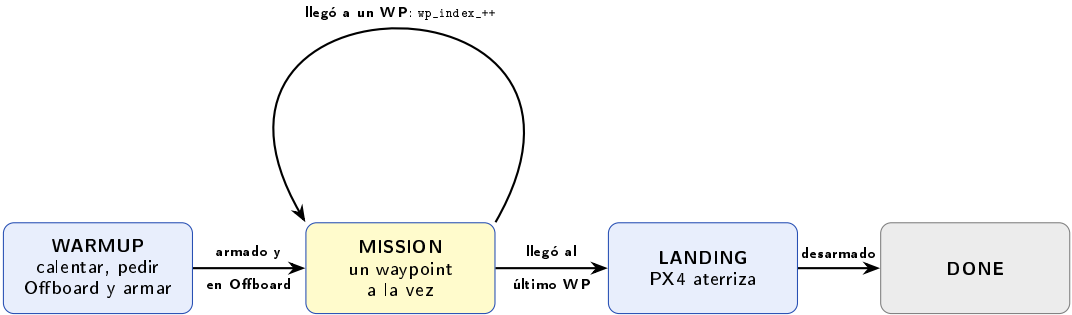



```cpp
// {norte, este, abajo}, todos a 5 m de altura
std::vector<std::array<float, 3>> waypoints_ = {
  {0.0f, 0.0f, -5.0f},   // WP0: subir
  {5.0f, 0.0f, -5.0f},   // WP1: 5 m al norte
  {5.0f, 5.0f, -5.0f},   // WP2: 5 m al este
  {0.0f, 5.0f, -5.0f},   // WP3: 5 m al sur
  {0.0f, 0.0f, -5.0f},   // WP4: de regreso
};
```

Código 1: square_mission.cpp: la lista de waypoints

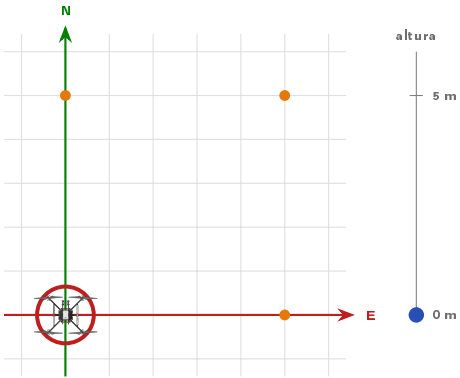
Un cuadrado de 5 m. El nodo manda un waypoint a la vez y pasa al siguiente cuando llega.

square_mission: la máquina de estados



- El nodo recuerda **en qué parte va** con `state_`. Cada 100 ms, `loop()` hace lo que toca según el estado (un `switch`).
- En **WARMUP** y **MISSION** publica heartbeat y setpoint. En **LANDING** ya no: le pidió a PX4 que aterrice (`NAV_LAND`) y PX4 lo hace solo.
- Casi todas las misiones (y los retos) tienen esta forma.

square_mission cuadro por cuadro



WARMUP → MISSION → LANDING → DONE

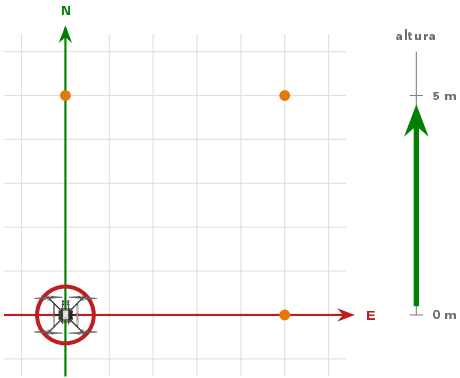
Publica: `position = {0, 0, -5}`

square_mission

```
arming=1 nav=4 alt=0.00 m wp=0
```

En tierra, calentando: el setpoint ya es WP0 (5 m arriba del despegue).

square_mission cuadro por cuadro



WARMUP | MISSION | LANDING | DONE

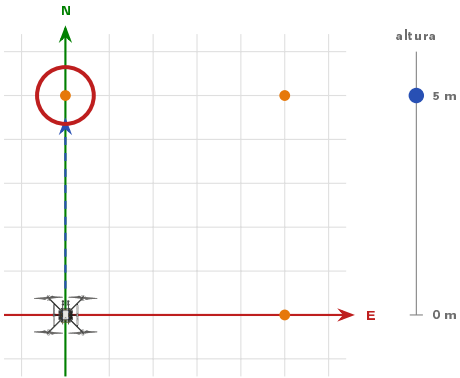
Publica: `position = {0, 0, -5}`

square_mission

```
Offboard activo, iniciando mision
arming=2 nav=14 alt=2.31 m wp=0
```

Armado y en Offboard: pasa a MISSION. Sube rumbo a WP0.

square_mission cuadro por cuadro



WARMUP | MISSION | LANDING | DONE

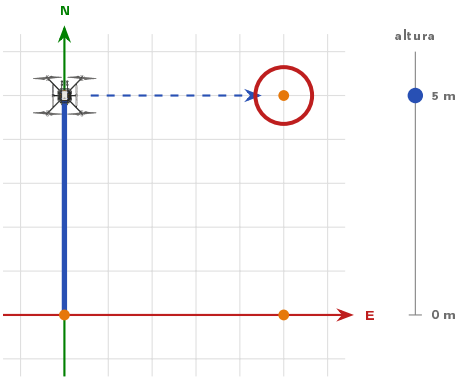
Publica: `position = {5, 0, -5}`

square_mission

Llegue al WP 0

A 5 m: `reached()` da verdadero, `wp_index_` pasa a 1 y el setpoint cambia a WP1. El dron va al **norte**.

square_mission cuadro por cuadro



WARMUP | MISSION | LANDING | DONE

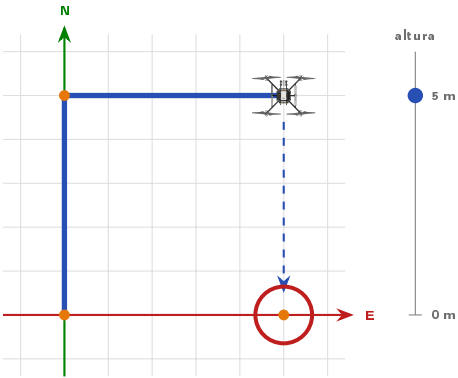
Publica: `position = {5, 5, -5}`

square_mission

Llegue al WP 1

Ahora al **este**. Fíjate: la nariz sigue apuntando al norte.

square_mission cuadro por cuadro



WARMUP | MISSION | LANDING | DONE

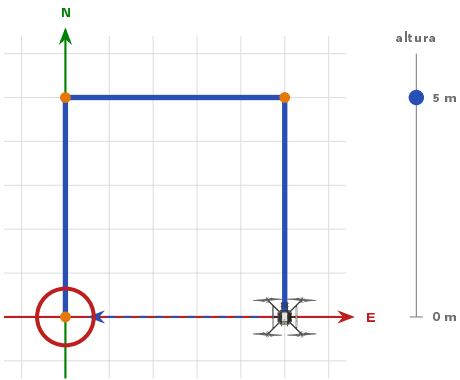
Publica: `position = {0, 5, -5}`

square_mission

Llegue al WP 2

Al **sur**: vuela de reversa.

square_mission cuadro por cuadro



WARMUP | MISSION | LANDING | DONE

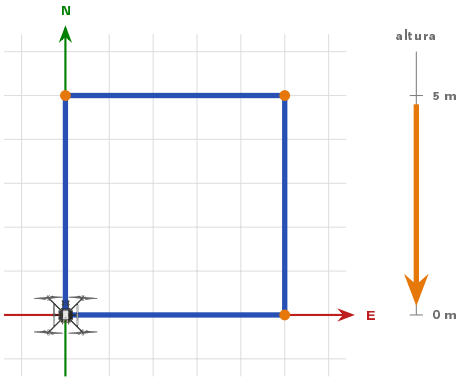
Publica: `position = {0, 0, -5}`

square_mission

Llegue al WP 3

Al **oeste**, de regreso al punto de despegue.

square_mission cuadro por cuadro



WARMUP | MISSION | LANDING | DONE

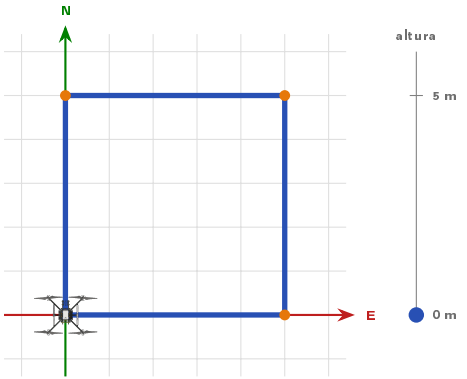
Publica: nada: aterriza PX4

square_mission

```
Llegue al WP 4
Mision completa, aterrizando
```

Era el último: manda `NAV_LAND` y deja de publicar setpoints. PX4 baja solo (`nav=18`).

square_mission cuadro por cuadro



WARMUP → MISSION → LANDING → DONE

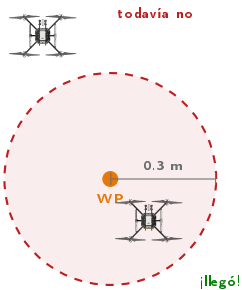
Publica: nada: aterriza PX4

square_mission

```
Aterrizado
Aterrizado
...
```

PX4 desarma al tocar tierra: DONE. (¿Por qué imprime "Aterrizado" 10 veces por segundo? Es el primer ejercicio.)

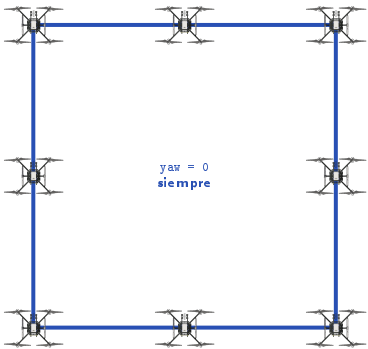
¿Cuándo "llegó"?




```
bool reached(const std::array<float, 3> & target) {
  float dx = position_.x - target[0];
  float dy = position_.y - target[1];
  float dz = position_.z - target[2];
  float distance = std::sqrt(dx*dx + dy*dy + dz*dz);
  return distance < 0.3f;
}
```

- Distancia en 3D entre donde está y donde quiere estar.
- Una **esfera** de 0.3 m: el dron nunca queda exacto, así que se pide "suficientemente cerca".
- Muy chica: nunca llega. Muy grande: corta las esquinas.

¿Por qué el dron no gira?



- `setpoint.yaw = 0.0f` en todos los setpoints: la nariz **siempre al norte**.
- Un multicóptero puede ir de lado y de reversa sin problema.
- Para que mire hacia donde va:
  `yaw = atan2(este, norte)` del tramo que sigue.

Así lo hace `figure_eight`. Con una cámara al frente, esto importa: ves lo que viene.

Pruébalo tú

```
vespo up --sim                                  # terminal 1
ros2 run px4_examples flight_mode_monitor       # terminal 2: el narrador
ros2 run px4_examples square_mission            # terminal 3
```

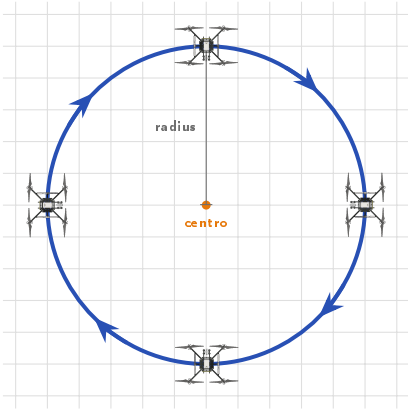
Ejercicios

1. Que "Aterrizado" se imprima **una sola vez**.
2. Un **triángulo** en vez de cuadrado.
3. Un cuadrado de 10 m **a 8 m** de altura.
4. Que la nariz **mire hacia donde va**.

Después de cada cambio

`vespo build px4_examples` y vuelve a correr el nodo. La simulación puede seguir prendida: si el dron quedó en el aire, **Land** en QGC antes de correrlo otra vez.

orbit_mission: un círculo con parámetros

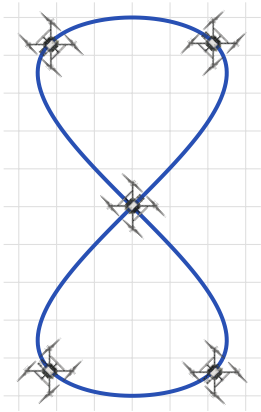


- Va de vuelta en vuelta en sentido **horario**, con la nariz siempre **al centro**.
- Manda **posición + velocidad**: dónde estar en el círculo y la velocidad tangente, para que no vaya a tirones de punto en punto.
- Todo se cambia sin compilar, con **parámetros**:

```
ros2 run px4_examples orbit_mission --ros-args \
  -p radius:=8.0 -p speed:=3.0 -p laps:=1
```

Parámetros: `center_x`, `center_y`, `altitude`, `radius`, `speed`, `laps`.

figure_eight: un 8 mirando hacia adelante



- La curva es una fórmula del tiempo:
  norte $= A\sin(\omega t)$, este $= \frac{A}{2}\sin(2\omega t)$.
- Además de la posición manda **velocidad y aceleración** (las derivadas): PX4 sabe hacia dónde va la curva y no se queda atrás.
- El yaw sigue la velocidad: la nariz **mira hacia adelante**.

```
ros2 run px4_examples figure_eight --ros-args \
  -p size:=8.0 -p period:=30.0
```

Los cinco de offboard

Ejemplo	Manda	Lo nuevo
`takeoff_mission`	posición fija	la receta: heartbeat, setpoint, modo y armar
`square_mission`	una lista de posiciones	máquina de estados, `reached()`
`orbit_mission`	posición + velocidad	parámetros de ROS 2, yaw al centro
`figure_eight`	posición + velocidad + aceleración	trayectoria como fórmula del tiempo
`velocity_control`	solo velocidad (y giro)	"adelante / derecha" del dron a NED

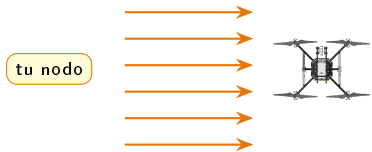
Ojo

En offboard **tu nodo es el piloto automático**: si tiene un bug, el dron hace el bug. Antes de probar algo nuevo, ten QGC a la mano para darle **Hold** o **Land**.

Contenido

1. Instalar todo con vespo
2. Repaso: el x500 y PX4
3. Cómo habla ROS 2 con PX4
4. shared_ptr: lo único de C++ que hoy hace falta
5. Escuchar al dron
6. Volar en offboard
7. Volar con comandos
8. Tu primer reto: la estrella del show
9. Cuando algo no jala

Offboard contra comandos



Offboard: “sigue mi mano”

Tu nodo manda dónde estar **10 veces por segundo**. Tú diseñas la trayectoria. Si te callas, PX4 pasa a Hold.



Comandos: “ve a tal lugar”

Tu nodo manda **una orden** (despega, ve a este punto GPS, regresa) y PX4 la ejecuta con sus propias maniobras. Si te callas, sigue con lo último que le pediste.

Comandos = lo mismo que los botones de QGC, pero desde tu código. Más simple y más seguro; menos control fino.

VehicleCommand: un formulario con 7 casillas

VehicleCommand

`command`	`NAV_TAKEOFF`
`param1`	
`param2`	
`param3`	
`param4`	
`param5`	latitud
`param6`	longitud
`param7`	altura (AMSL)

- `command` dice **qué hacer**; los 7 `param` son los detalles. Cada comando usa los suyos.
- Una casilla que no aplica va en `NaN` ("no me importa") o en 0.

Comando	Casillas
`COMPONENT_ARM_DISARM`	p1: 1 armar, 0 desarmar
`DO_SET_MODE`	p1 = 1, p2: modo (6 = Offboard)
`NAV_TAKEOFF`	p5–p7: dónde y a qué altura
`DO_REPOSITION`	p1 velocidad, p5–p7: a dónde
`NAV_LAND`	ninguna
`NAV_RETURN_TO_LAUNCH`	ninguna

¿Me hizo caso? El ack

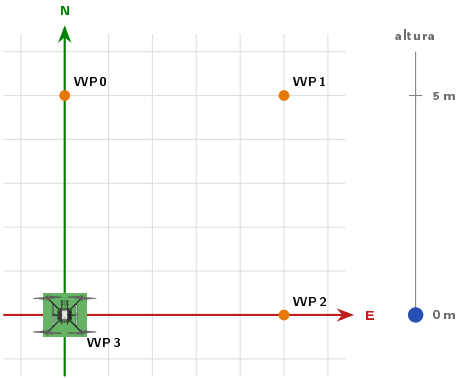


goto_mission

```
PX4 rechazó el comando 400 (result=1)
```

- PX4 contesta cada comando. `result = 0` es **aceptado**; otro número, rechazado.
- 400 = armar. ¿Por qué lo rechazó? QGC y `flight_mode_monitor` dicen la razón.
- Los ejemplos con comandos **reintentan** cada segundo: al arrancar, PX4 a veces aún no está listo.

goto_mission paso a paso



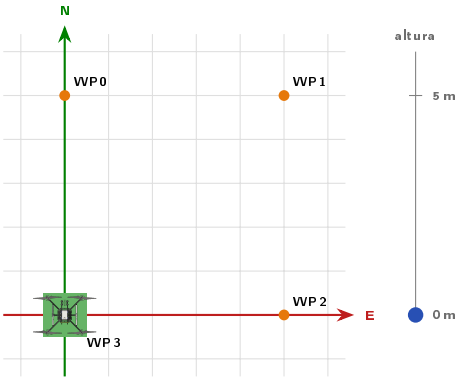
WAIT_GPS | ARMING | TAKEOFF | GOTO | RETURN | DONE

goto_mission

```
goto_mission: esperando posición GPS...
GPS listo, armando
```

Espera a que PX4 sepa dónde está (latitud y longitud válidas). Guarda ese punto como **casa**.

goto_mission paso a paso



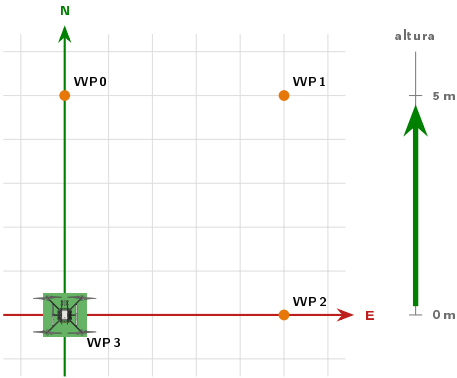
WAIT_GPS → ARMING → TAKEOFF → GOTO → RETURN → DONE

goto_mission

```
(cada segundo: Hold, luego armar)
```

Pide modo **Hold** y luego **armar**, y reintenta cada segundo hasta que PX4 dice que sí. Nada de setpoints: aquí no hay Offboard.

goto_mission paso a paso



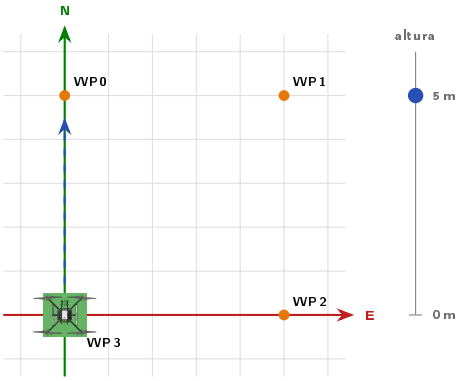
WAIT_GPS | ARMING | TAKEOFF | GOTO | RETURN | DONE

goto_mission

```
Armado, despegando a 5.0 m
```

Un solo `NAV_TAKEOFF`. PX4 sube solo y al llegar se queda en Hold.

goto_mission paso a paso



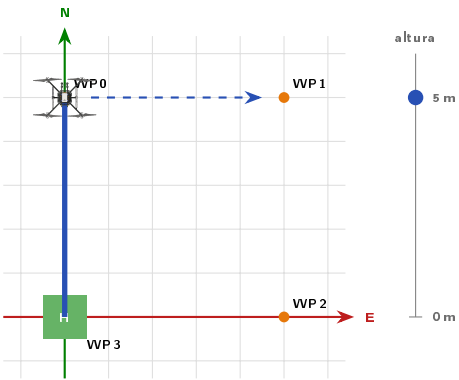
WAIT_GPS | ARMING | TAKEOFF | GOTO | RETURN | DONE

goto_mission

```
Yendo al WP 0: 5.0 m N, 0.0 m E
WP 0 a 3.2 m
```

Convierte “5 m al norte” a latitud y longitud y manda un `DO_REPOSITION`. PX4 vuela hasta allá.

goto_mission paso a paso



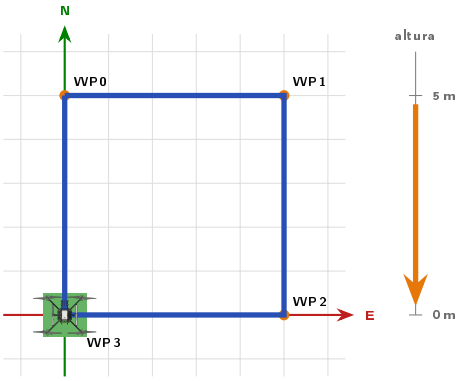
WAIT_GPS | ARMING | TAKEOFF | **GOTO** | RETURN | DONE

goto_mission

```
Llegué al WP 0
Yendo al WP 1: 5.0 m N, 5.0 m E
```

Mide la distancia con la posición local; a menos de 0.5 m manda el siguiente. Una orden por waypoint, no 10 por segundo.

goto_mission paso a paso



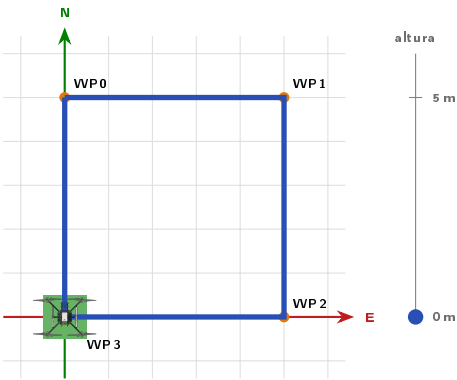
WAIT_GPS | ARMING | TAKEOFF | GOTO | **RETURN** | DONE

goto_mission

```
Llegué al WP 3
Misión completa, regresando a casa
```

Al terminar, `NAV_RETURN_TO_LAUNCH`: PX4 regresa y aterriza solo.

goto_mission paso a paso



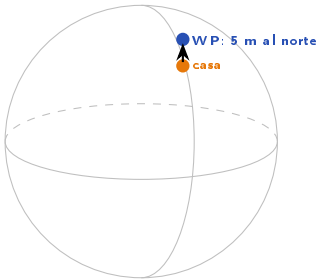
WAIT_GPS | ARMING | TAKEOFF | GOTO | RETURN | DONE

goto_mission

En casa y desarmado

PX4 desarmó al tocar tierra. Fin.

De metros a GPS



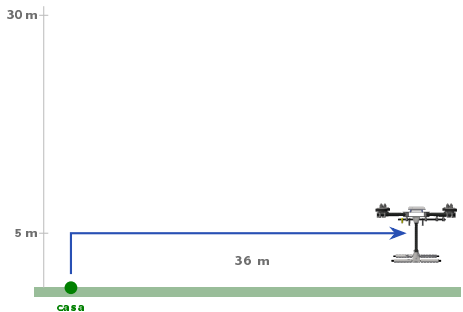
`DO_REPOSITION` pide **latitud y longitud**, no metros. Desde casa:

$$\text{lat} = \text{lat}_0 + \frac{\text{norte}}{R} \cdot \frac{180}{\pi} \qquad \text{lon} = \text{lon}_0 + \frac{\text{este}}{R\cos(\text{lat}_0)} \cdot \frac{180}{\pi}$$

con $R = 6\,371\,000$ m (radio de la Tierra).

- 1 grado de latitud ≈ 111 km: 5 m son ~0.000045°.
- Por eso la latitud se guarda en `double`: en `float` se pierden los metros.

return_home: aléjate y regresa

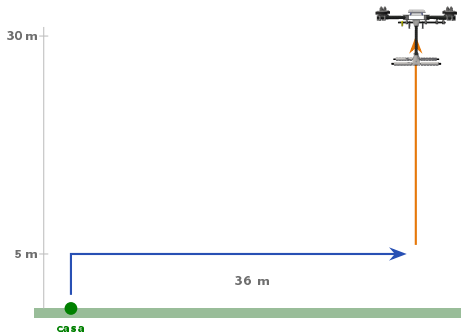


return_home

```
Alejándome 30 m N, 20 m E
```

Despega a 5 m y se aleja con `DO_REPOSITION`.

return_home: aléjate y regresa

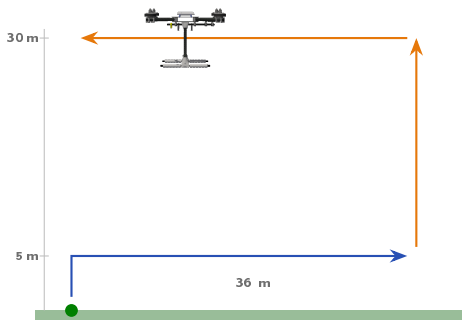


return_home

```
Lejos de casa (36 m). Return!
subiendo a la altura de retorno
| a 35.8 m de casa, altura 12.4 m
```

Manda `NAV_RETURN_TO_LAUNCH`. Lo primero que hace PX4 es **subir** a la altura de retorno (30 m en el x500 de SITL) para no chocar con nada.

return_home: aléjate y regresa

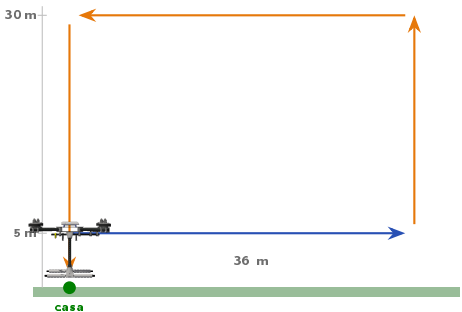


return_home

```
volando a casa | a 21.4 m de casa,
altura 30.0 m
```

Vuela en línea recta hacia casa, a esa altura.

return_home: aléjate y regresa



return_home

```
sobre casa, bajando | a 0.4 m de casa,
altura 8.7 m
En casa y desarmado (a 0.3 m del home)
```

Baja, aterriza y desarma solo. Es lo mismo que pasa con batería baja o si se pierde la conexión.

Los otros comandos

orbit_command
PX4 vuela el círculo solo (`DO_ORBIT`). Compáralo con `orbit_mission`, donde lo calculabas tú.

change_speed
Cambia la velocidad en pleno vuelo (`DO_CHANGE_SPEED`).

set_home
Mueve la "casa" a otro punto: Return regresa ahí (`DO_SET_HOME`).

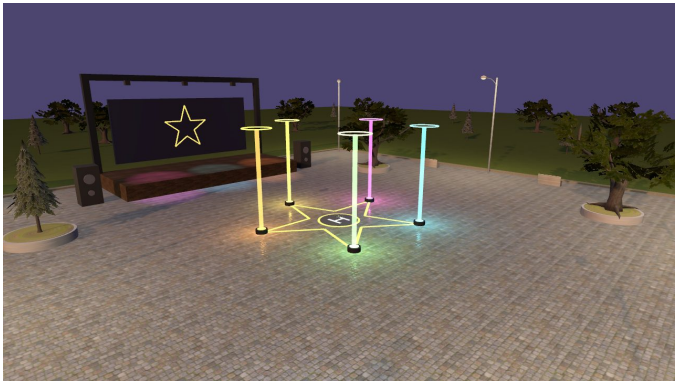
motor_test
Prueba un motor a la vez, en tierra (`ACTUATOR_TEST`). En el real: **sin hélices**.

¿Offboard? trayectorias tuyas, reaccionar rápido a sensores (seguir algo, esquivar).
¿Comandos? ir de un punto a otro, despegar, aterrizar, regresar. Si un comando hace lo que necesitas, úsalo.

Contenido

1. Instalar todo con vespo
2. Repaso: el x500 y PX4
3. Cómo habla ROS 2 con PX4
4. shared_ptr: lo único de C++ que hoy hace falta
5. Escuchar al dron
6. Volar en offboard
7. Volar con comandos
8. Tu primer reto: la estrella del show
9. Cuando algo no jala

La misión

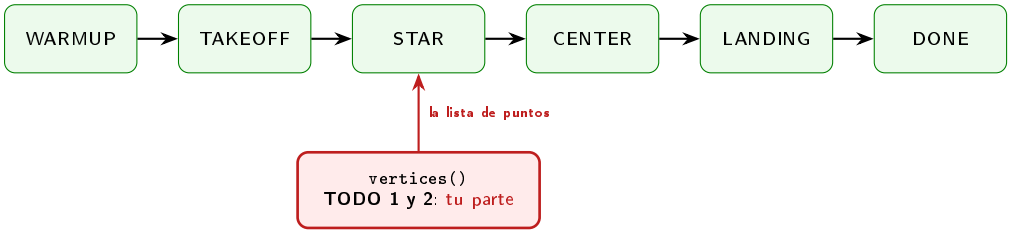


El mundo `plaza_show` de `x500_retos` (render propio)

Año nuevo en la plaza. El dron tiene que **dibujar una estrella en el cielo** pasando por los aros de luz.

- Mundo: `plaza_show`.
- Nodo: `estrella_show`, de `x500_retos`.
- Viene de `square_mission` (waypoints en offboard) y `orbit_mission` (parámetros).
- **Lo nuevo**: los waypoints ya no se escriben a mano, se **calculan con una fórmula**.

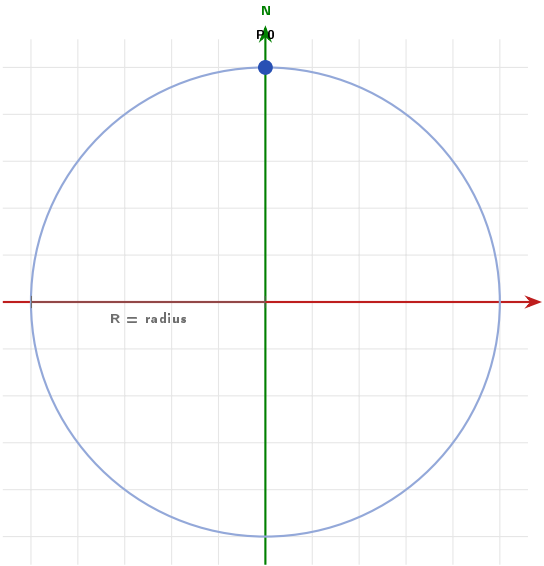
Qué ya está hecho y qué te toca



- En verde: ya funciona. Despega, recorre la lista, regresa al centro, aterriza y te dice qué tan bien lo hiciste.
- En rojo: `vertices()` regresa la lista de puntos de la estrella. Hoy regresa una lista **vacía**.
- Si lo corres sin hacer nada, despega, te avisa y aterriza:

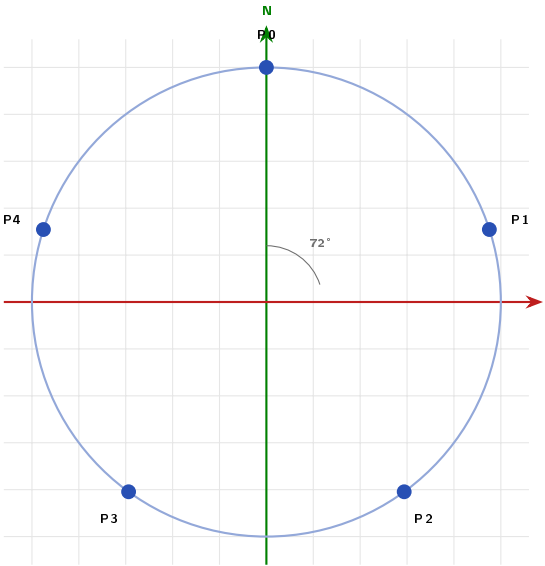
```
ros2 run x500_retos estrella_show
vertices() regresó una lista vacía: falta el TODO. Aterrizando
```

La geometría, paso a paso



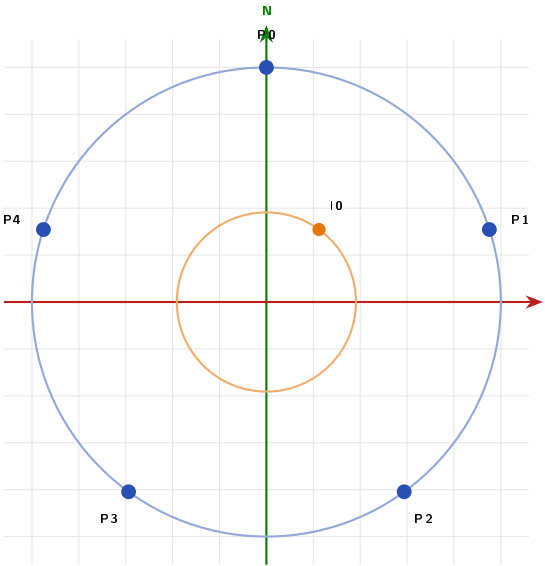
1. **El círculo**. Todos los picos están en un círculo de radio `radius` (5 m). El primero, **P0**, al norte.

La geometría, paso a paso



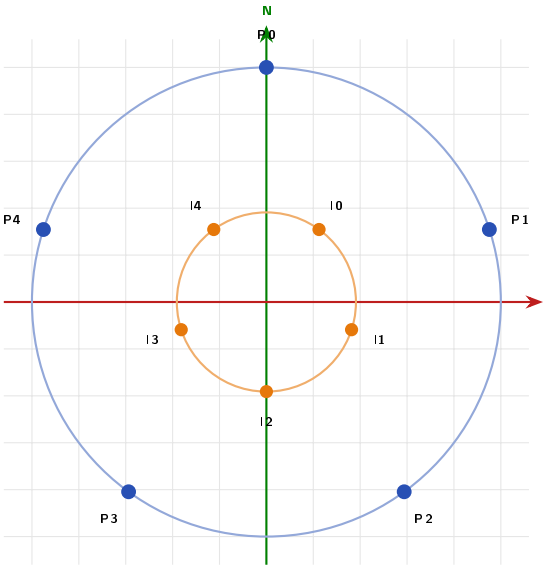
2. Los picos. Con `points` = 5 picos, entre uno y otro hay $360/5 = 72$. El pico k está a un ángulo $\frac{2\pi k}{\texttt{points}}$ (en radianes) medido desde el norte hacia el este. **Esto es el TODO 1**.

La geometría, paso a paso



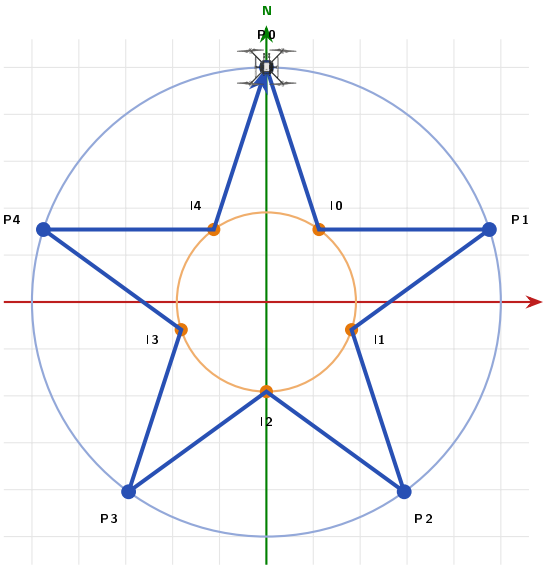
3. El círculo de adentro. Los vértices interiores están en un círculo más chico: radio `radius * inner_ratio` ($5 \times 0.382 = 1.91$ m). **I0** va a la mitad, en ángulo, entre P0 y P1.

La geometría, paso a paso



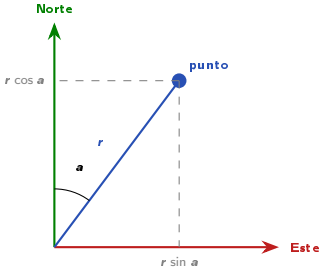
4. **Todos los interiores.** Uno después de cada pico. **Esto es el TODO 2.** Si `inner_ratio` es 0, no hay interiores.

La geometría, paso a paso



5. El orden de vuelo. P0, I0, P1, I1, . . . , P4, I4 y de vuelta. Esa es la lista que `vertices()` tiene que regresar; el resto del nodo la vuela.

De ángulo a norte y este



Un punto a distancia r y ángulo a (desde el norte, girando hacia el este):

$$\text{norte} = r\cos a \qquad \text{este} = r\sin a$$

En el código, cada vértice es `{norte, este, z}` con `z = -altitude` y se agrega con `out.push_back({norte, este, z});`

Ojo

- `std::cos` y `std::sin` usan **radianes**, no grados.
- `k / points_` con dos enteros da **0**: multiplica primero por `2 * M_PI`.
- `radius_` es `double` y el punto lleva `float`: `static_cast<float>(...)`.

Pruébalo

```
vespo up --sim --world plaza_show                    # terminal 1
ros2 run x500_retos estrella_show                    # terminal 2
# ... cambias vertices() en src/x500_retos/src/estrella_show.cpp ...
vespo build x500_retos                               # y otra vez ros2 run
```

al final, si salió bien

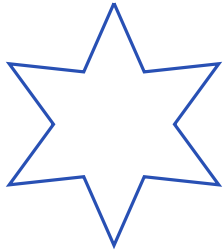
```
Aterrizado. Qué tan cerca pasó de cada vértice:
1: 0.08 m OK
2: 0.11 m OK
...
10: 0.09 m OK
Misión cumplida
```

Se cumple si pasa a menos de **0.5 m** de cada vértice, regresa al centro y aterriza. Si uno dice FALLÓ, ese punto quedó mal calculado.

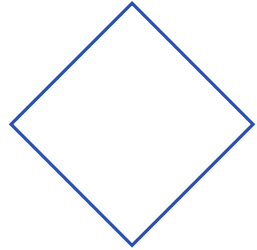
Si tu fórmula es correcta, sirve para todo



por defecto
`points:=5`



`points:=6`
`inner_ratio:=0.5`



`points:=4`
`inner_ratio:=0.0`: ¡un cuadrado!

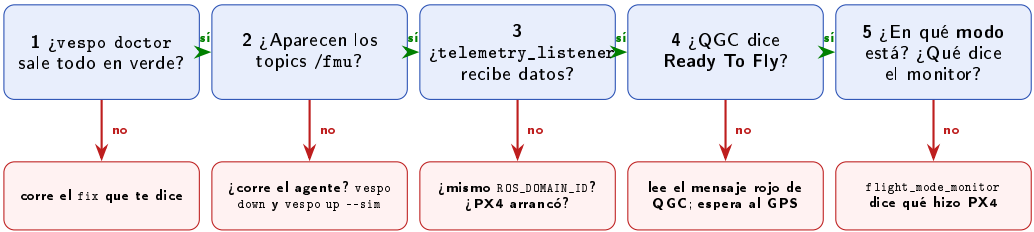
```
ros2 run x500_retos estrella_show --ros-args -p points:=6 -p inner_ratio:=0.5
ros2 run x500_retos estrella_show --ros-args -p points:=4 -p inner_ratio:=0.0
```

Con 4 picos y sin interiores es el `square_mission`, pero calculado: el mismo código hace las tres figuras.

Contenido

1. Instalar todo con vespo
2. Repaso: el x500 y PX4
3. Cómo habla ROS 2 con PX4
4. shared_ptr: lo único de C++ que hoy hace falta
5. Escuchar al dron
6. Volar en offboard
7. Volar con comandos
8. Tu primer reto: la estrella del show
9. Cuando algo no jala

Revisa en este orden



Casi siempre el problema está en el **primer** paso que falla. No te saltes pasos: revisar tu código cuando el agente ni siquiera corre es tiempo perdido.

Síntoma, causa, arreglo (1/2)

Ves	Casi siempre es	Haz
`ros2 topic list` no tiene /fmu	el agente no corre	`vespo up --sim` (o `vespo agent`)
Tu nodo no recibe nada, sin error	nombre sin `_v1`, o QoS `reliable`	copia nombre y QoS de un ejemplo
`No executable found`	no compilaste, o paquete nuevo	`vespo build` y terminal nueva
Cambiaste el código y hace lo mismo	no compilaste (o falló el build)	`vespo build`, lee los errores
`UDP port 8888 busy`	quedó algo de antes	`vespo down`
Gazebo lentísimo o la laptop se apaga	batería, o un proceso zombi	cargador; `vespo down`

Síntoma, causa, arreglo (2/2): el dron

Ves	Casi siempre es	Haz
No arma: “No connection to the GCS”	QGroundControl cerrado	ábrelo (`vespo qgc`); `vespo up` ya lo abre
No arma recién arrancado	el GPS o el estimador aún no están listos	espera a **Ready To Fly**
No entra a Offboard	no hubo setpoints antes	calienta 1 s antes de pedir el modo
Sube y luego pasa a **Hold**	tu nodo dejó de publicar (Ctrl+C, crash, un `sleep`)	`flight_mode_monitor` lo confirma
Se va a otro lado	confundiste NED: z positivo es **abajo**; x es norte	revisa signos y orden
Llega y nunca pasa al siguiente WP	tolerancia muy chica o WP inalcanzable	imprime la distancia al WP
Se desarma solo en tierra	armó y nadie le pidió despegar	normal: es seguridad de PX4

La hoja para tener a la mano

```
vespo doctor           # todo bien?
vespo update           # lo nuevo del equipo
vespo build [PKG]      # compilar
vespo up --sim         # simulacion completa
vespo up --sim --world plaza_show
vespo down             # apagar todo
vespo simulate --show  # modelos y mundos
vespo config           # mi configuracion
```

Código 2: vespo

```
ros2 run px4_examples telemetry_listener
ros2 run px4_examples flight_mode_monitor
ros2 topic list | grep fmu
ros2 topic echo /fmu/out/vehicle_status_v1 \
  --field nav_state
ros2 interface show px4_msgs/msg/<Tipo>
```

Código 3: ROS 2

nav_state: 4 Hold · 14 Offboard · 17 Takeoff · 18 Land · 5 Return

Y después

1. Corre los 18 ejemplos, con `flight_mode_monitor` al lado.
2. Haz los ejercicios de `square_mission`.
3. Resuelve la **estrella**.
4. Sigue con los retos de `x500_retos`: el pilar, la entrega, la geocerca, la cueva...

¿Dudas de un mensaje o de un ejemplo línea por línea? Está todo en la **referencia técnica** (`referencia.pdf`).



Créditos de las imágenes



- **Capturas de QGroundControl y Gazebo, fotos y el diagrama de uXRCE-DDS**: documentación de PX4 (`docs.px4.io`), © PX4 / Dronecode y colaboradores, licencia **CC BY 4.0** (`creativecommons.org/licenses/by/4.0`). Solo se redimensionaron.
- **Renders del x500 y de los mundos**: propios, con Gazebo Harmonic sobre el modelo `x500` de PX4 (`tools/renders_x500.py`).

